

PROJECT REPORT

Smart Campus Vision & Issue Vigilance System

An AI-Assisted Campus Facility Complaint & Defect Inspection Platform

Domain: Full-Stack Web Development, Computer Vision & Applied AI

Technology Stack: Python 3.12 · Flask 3.0 · OpenCV 4.11 · SQLite3 · Pytest

Submitted as part of academic / portfolio project requirements

Project Repository: campus complaint system

Date: September 2026

1. Introduction

Modern university campuses generate a continuous stream of facility maintenance issues — cracked walls, broken furniture, faulty lighting, plumbing leaks, and cleanliness lapses. Left to purely manual, paper-based or ad-hoc digital reporting, such issues are frequently mis-described, duplicated, or under-prioritised, resulting in safety hazards and dissatisfied students. The Smart Campus Vision & Issue Vigilance System is a full-stack web application that digitises this entire workflow and layers an automated Computer Vision (CV) inspection pipeline on top of it, so that every complaint submitted with a photograph is objectively analysed for image quality, defect severity, and recommended priority before it ever reaches a human administrator.

The platform serves three categories of stakeholders: students who report issues, facility administrators who triage and resolve them, and (indirectly) department heads who consume aggregate analytics. It combines a Flask-based backend, a SQLite relational store, a responsive HTML/CSS/JS frontend, and an OpenCV-driven vision service that performs blur detection, adaptive edge and contour analysis, and severity scoring on every uploaded image.

1.1 Purpose of this Report

This document presents the complete technical account of the project — its motivating problem, requirements, architecture, design artefacts (UML and ER diagrams), implementation details, testing strategy, and the learnings and future roadmap that emerged from building it. It is intended to serve both as an academic project submission and as living technical documentation for the codebase.

1.2 Project Snapshot

Attribute	Detail
Project Name	Smart Campus Vision & Issue Vigilance System
Category	Full-Stack Web Application with integrated Computer Vision
Backend	Python 3.12, Flask 3.0 (Blueprint architecture)
Frontend	HTML5, CSS3 (Flexbox/Grid), Vanilla JavaScript, Jinja2 templates
Computer Vision	OpenCV (cv2) 4.11.0, NumPy
Database	SQLite3 with foreign-key cascade enforcement
Security	Werkzeug password hashing, HMAC-based CSRF tokens, parameterized SQL
Testing	Pytest — 38 automated unit, integration and CV test cases
Reporting	ReportLab-generated PDF project report

2. Problem Statement

Traditional university campus complaint and facility-management systems suffer from persistent inefficiencies: manual and subjective reporting, ambiguous issue descriptions, an absence of objective visual

evidence, and zero automated prioritisation. Facilities personnel are routinely inundated with redundant, unverified, or poorly documented complaints, while students experience prolonged delays and a lack of transparency regarding the status of critical maintenance issues — structural wall cracks, electrical failures, lab-equipment breakage, or sanitation hazards among them.

Without an automated mechanism to validate photo clarity, assess physical defect severity, and categorise issues in real time, campus maintenance teams struggle to allocate resources effectively. This leads directly to safety hazards, accelerated asset depreciation, and diminished student satisfaction. The project therefore set out to answer a concrete question: can a lightweight, self-hosted web platform use classical computer-vision techniques — without requiring an expensive trained deep-learning model or cloud AI subscription — to inject objective, quantifiable evidence into a campus complaint pipeline?

2.1 Scope — In Scope

- Role-based access control and user management: secure student self-registration, credential authentication, session isolation, and administrative governance.
- Automated CV defect inspection: image-quality/blur detection via Laplacian variance; adaptive Canny edge detection, Otsu thresholding and morphological contour analysis; a 0–100% Defect Severity Score with automated priority inference (Low / Medium / High); and generation of annotated inspection images with bounding boxes and a metadata HUD banner.
- Complaint lifecycle management (CRUD): submission, tracking, editing (for unresolved tickets), and deletion with student-ownership safeguards.
- Administrator governance: real-time filtering, keyword search, visual-proof review, status transitions (Pending → In Progress → Resolved), and timestamped administrative remarks.
- Analytics and reporting dashboard: totals by status, category distribution, and CV verification metrics.
- Security and integrity controls: CSRF protection with HMAC verification, PBKDF2/SHA-256 password hashing, parameterised SQLite queries, and sanitised file uploads.

2.2 Scope — Out of Scope (Future Boundaries)

- Direct integration with enterprise ERP / SAP campus procurement portals.
- Real-time GPS telemetry from drone-based aerial campus surveys (reserved for a future phase).
- Native compiled iOS/Android binaries — the system is instead delivered as a fully responsive web application.

2.3 Target Users & Personas

Persona	Needs	Expectations
University Students (Primary)	Quick, friction-free reporting of classroom, hostel, or campus infrastructure defects from any device.	Visual feedback on the uploaded photo, real-time status updates, visibility into admin remarks.
Facility Administrators (Secondary)	A centralised triage dashboard sorting issues by severity score and category.	Filter queues by status, attach remarks, monitor overall campus maintenance health.
Department Heads (Tertiary)	High-level statistics on maintenance bottlenecks and resolution throughput.	Aggregate dashboards rather than individual ticket detail.

3. Functional Requirements

The functional requirements are grouped by module and traced to the implemented routes/controllers.

ID	Requirement	Module
FR-01	The system shall allow a new student to self-register with name, email and password, with server-side validation of email format and minimum password length.	auth.py
FR-02	The system shall authenticate users via salted, hashed credentials and maintain a secure server-side session.	auth.py
FR-03	The system shall issue and validate a per-session CSRF token on every state-changing request using constant-time comparison (hmac.compare_digest).	app.py
FR-04	The system shall enforce role separation, routing Students and Administrators to distinct dashboards and blocking cross-role access.	app.py / admin.py
FR-05	The system shall allow a student to submit a complaint with title, category, description, priority and an optional photo via a multipart form.	complaints.py
FR-06	On photo upload, the system shall automatically reject images below a sharpness threshold and prompt the student to re-upload.	vision_service.py
FR-07	For images that pass the sharpness check, the system shall run edge/contour detection, compute a Defect Severity Score (0–100%), infer a priority level, and generate an annotated overlay image.	vision_service.py
FR-08	The system shall persist each complaint together with its original photo, its annotated CV overlay, and computed severity metadata.	models.py
FR-09	The system shall let a student view all of their own complaints with live status badges (Pending / In Progress / Resolved).	complaints.py
FR-10	The system shall allow a student to edit or delete a complaint they submitted, provided it has not already been resolved.	complaints.py
FR-11	The system shall allow an administrator to view all complaints, search by keyword, and filter by status or category.	admin.py
FR-12	The system shall let an administrator inspect a side-by-side comparison of the raw photo and its CV-annotated counterpart along with severity metrics.	admin.py
FR-13	The system shall allow an administrator to transition a complaint's status and attach a timestamped remark visible to the student.	admin.py
FR-14	The system shall present real-time KPI cards (Total, Pending, In Progress, Resolved) and CV statistics (verified-photo count, average severity).	reports.py
FR-15	The system shall present a category-wise distribution table with percentage share of total complaints.	reports.py

4. Non-Functional Requirements

Category	Requirement
Security	Passwords are hashed with Werkzeug's PBKDF2/SHA-256 implementation; sessions rely on signed, HTTP-only cookies; every mutating route is CSRF-protected with constant-time token comparison; all SQL is parameterised to prevent injection; uploaded files are extension- and content-checked before being written to disk.
Performance	CV inference (blur check, edge/contour extraction, overlay generation) completes within roughly 1–2 seconds per image on commodity hardware since only classical, non-deep-learning OpenCV operations are used.
Usability	The UI is a responsive, mobile-friendly layout (Flexbox/Grid) so students can file complaints from a phone camera without dedicated app installation.
Reliability	Database writes for a complaint and its two associated image paths are wrapped so a failure in image processing does not leave an orphaned or half-written database row.
Maintainability	The backend is organised into Flask Blueprints (auth, complaints, admin, reports) with a single Data Access Layer (models.py), keeping controllers thin and business/query logic centralised.
Testability	38 automated Pytest cases (route-level integration tests and CV unit tests) run against an isolated, disposable test database via fixtures in conftest.py.
Portability	The stack (Flask + SQLite + OpenCV) has no external service dependency, so the entire application runs locally with <code>python run.py</code> on any machine with Python 3.10+.
Auditability	Every complaint carries created/updated timestamps and administrator remarks, giving a lightweight audit trail of status changes.

5. System Architecture

The application follows a classical layered (N-tier) web architecture. A Flask application factory (app.py) wires together four Blueprints — auth, complaints, admin and reports — each of which delegates all persistence to a single Data Access Layer module (models.py) sitting atop a SQLite connection manager (database.py). Photo uploads are intercepted by a dedicated Computer Vision Engine (vision_service.py) built on OpenCV and NumPy, which runs independently of the request/response cycle logic and returns a structured result (severity score, priority, annotated image path) back to the complaints controller for persistence.

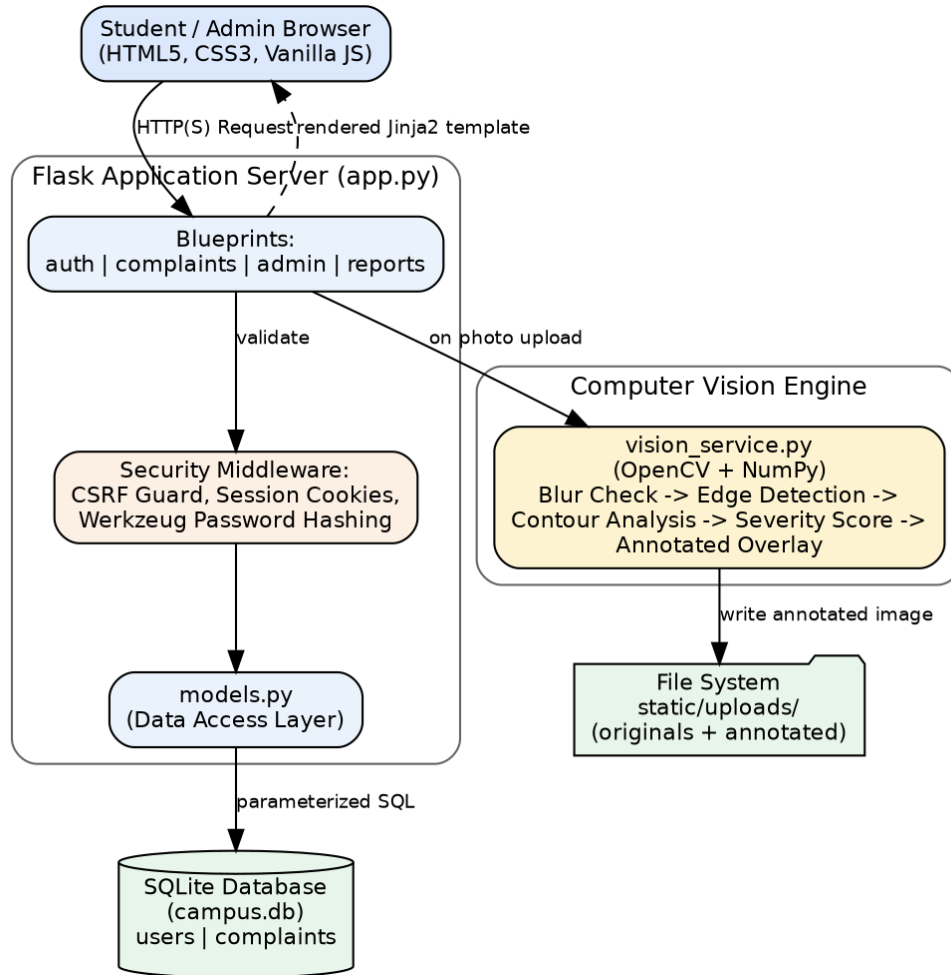


Figure 5.1 — High-level System Architecture

5.1 Layer Responsibilities

Layer	Responsibility	Key Files
Presentation	Renders Jinja2 templates, handles client-side form validation and progressive enhancement.	templates/*.html, static/js/script.js, static/css/style.css
Application / Routing	Flask Blueprints expose HTTP routes, enforce authentication/authorisation and CSRF, and orchestrate calls to the vision and data layers.	app.py, auth.py, complaints.py, admin.py, reports.py
Computer Vision Engine	Independent module performing blur filtering, edge/contour detection, severity scoring and overlay annotation.	vision_service.py
Data Access	Centralised SQL query construction and result mapping; the only layer allowed to touch the database directly.	models.py, database.py
Persistence	SQLite relational store and the filesystem for uploaded/annotated images.	database/campus.db, static/uploads/

Layer	Responsibility	Key Files
Configuration	Central, environment-agnostic settings: secret key, paths, allowed extensions.	config.py

5.2 Request Lifecycle (Summary)

1. Browser sends an HTTP request to a Flask route.
2. A before-request hook validates the CSRF token (for state-changing requests) and the session.
3. The relevant Blueprint controller executes business logic, invoking vision_service.py when a photo is present.
4. models.py executes parameterised SQL against database.py's connection.
5. The controller renders a Jinja2 template (or issues a redirect) back to the browser.

6. Design Diagrams

This section presents the five core design artefacts used to plan and document the system's structure and behaviour: a Use Case Diagram, a Workflow Diagram, a Sequence Diagram, a Class/Component Diagram, and an Entity–Relationship Diagram.

6.1 Use Case Diagram

The Use Case Diagram captures the two actors — Student and Facility Administrator — and the discrete goals each can accomplish. The Submit Complaint use case includes CV Defect Analysis automatically, while reviewing visual evidence extends the base Search/View use case for the administrator.

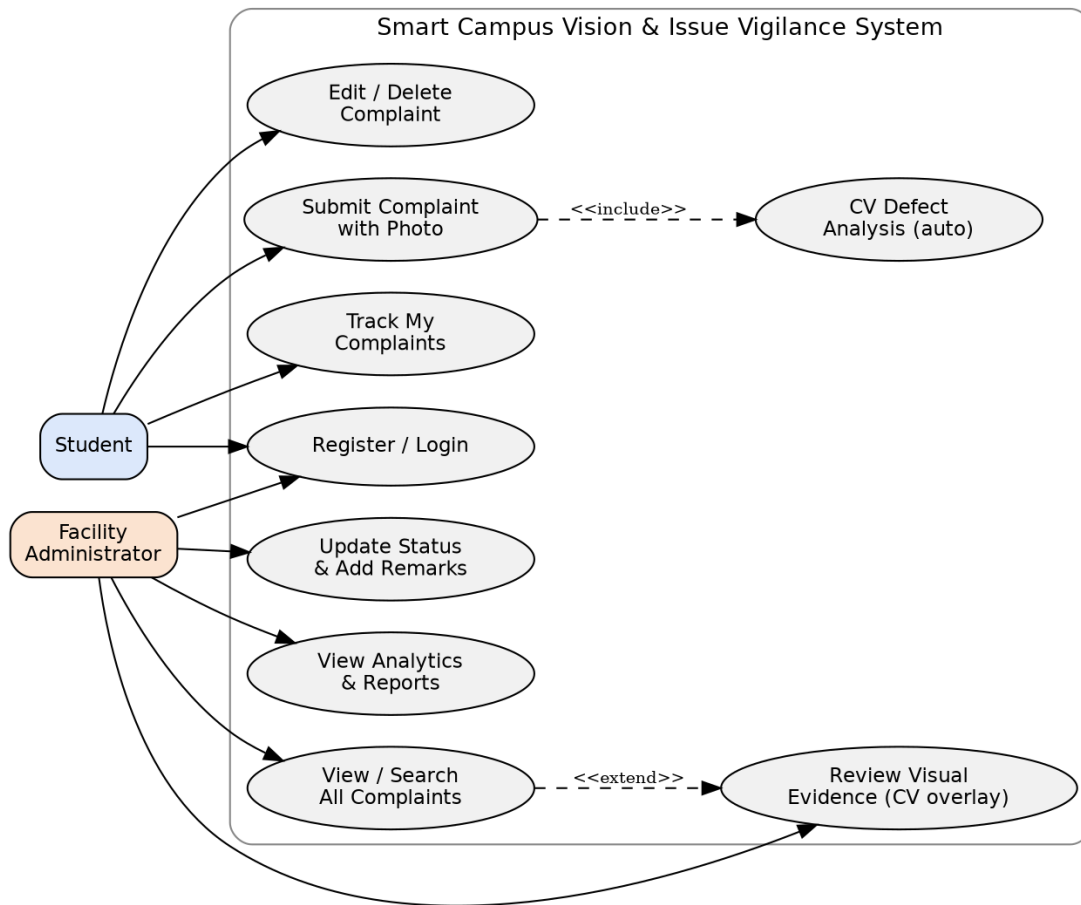


Figure 6.1 — Use Case Diagram

6.2 Workflow Diagram

The workflow diagram traces the full life of a single complaint — from student login through the CV inspection gate (which can reject and loop back on blurry photos) to administrator triage and final resolution notification.

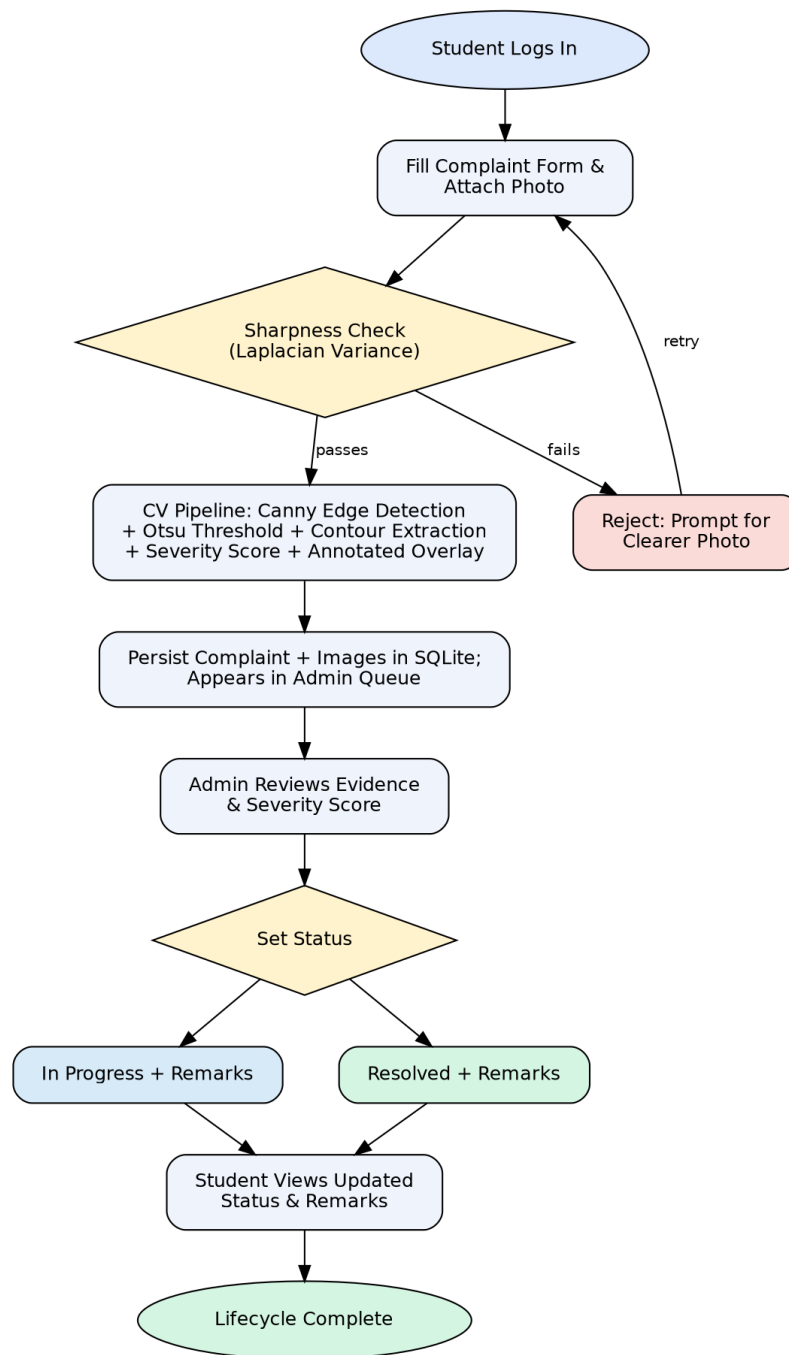


Figure 6.2 — End-to-End Complaint Workflow

6.3 Sequence Diagram

The sequence diagram below details the interaction between the Student's browser, the Flask complaints controller, the Vision Service, and the SQLite database during a single complaint submission that includes a photo.

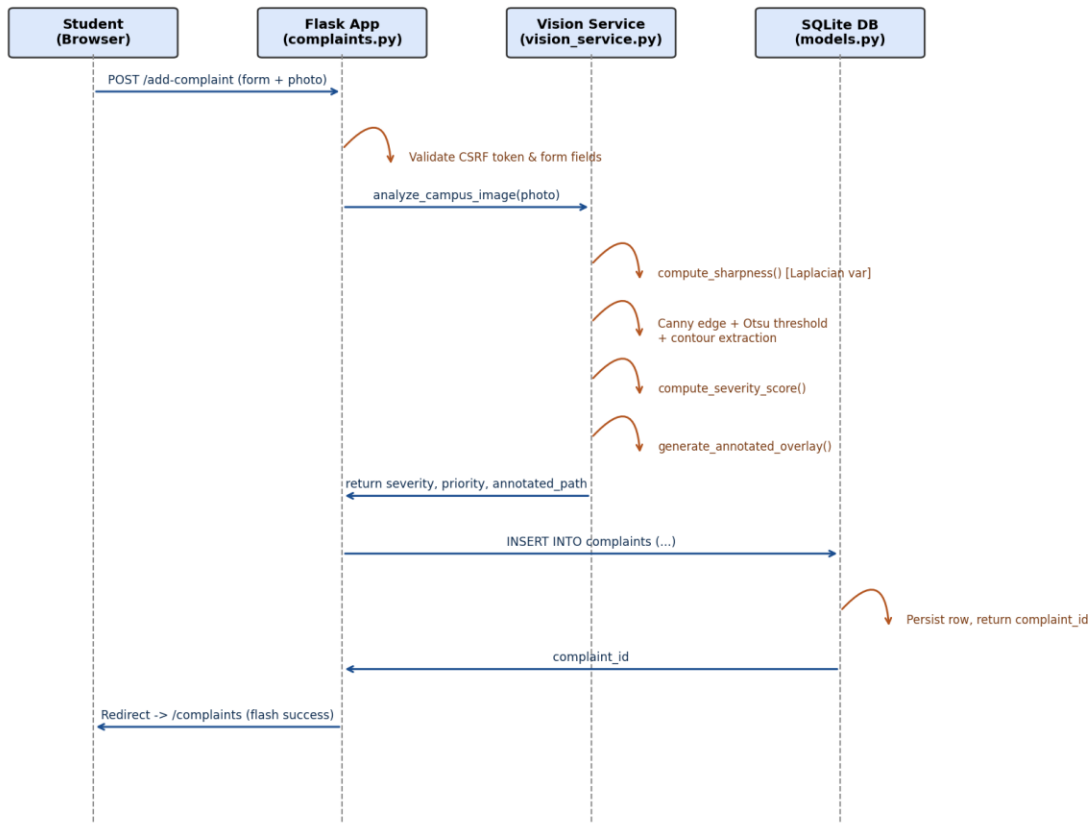
Sequence Diagram: Complaint Submission with CV Analysis

Figure 6.3 — Sequence Diagram: Complaint Submission with CV Analysis

6.4 Class / Component Diagram

The component diagram shows the module-level decomposition of the codebase: the Flask application factory registers four Blueprints, each of which depends on the shared Data Access Layer; the complaints Blueprint additionally depends on the Computer Vision Engine.

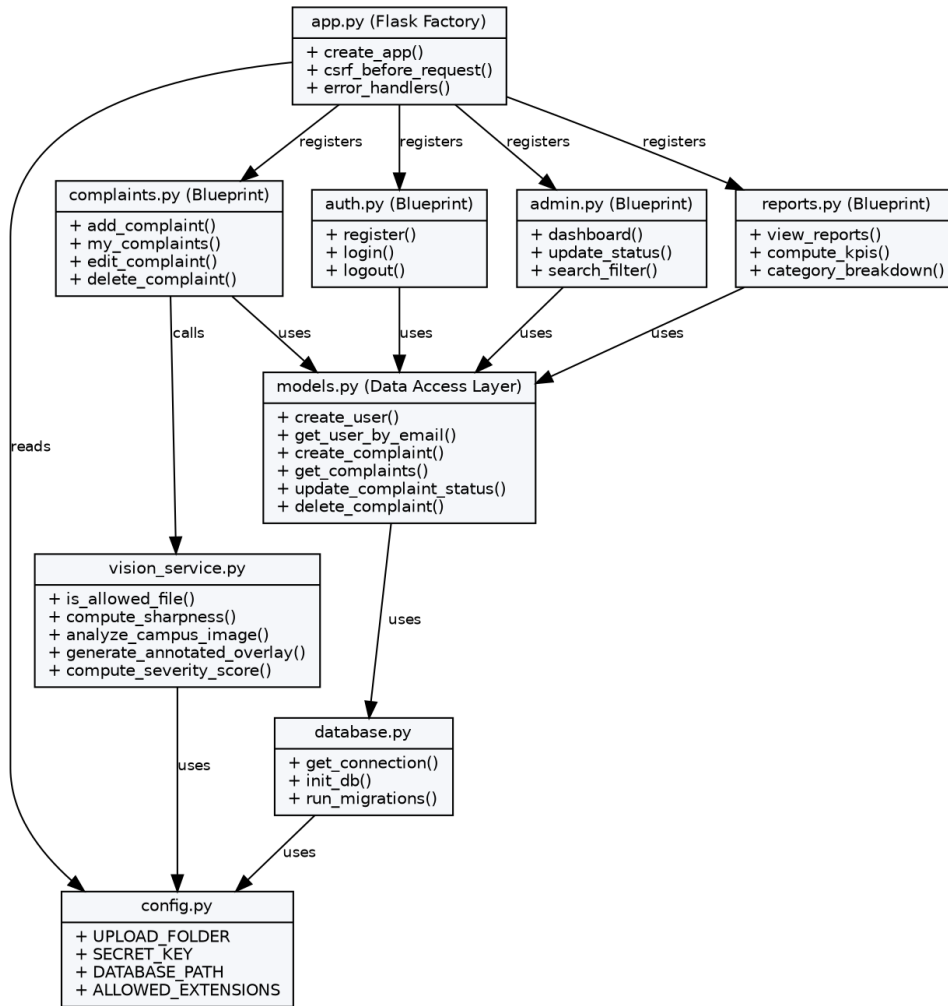


Figure 6.4 — Class / Component Diagram

6.5 Entity–Relationship (ER) Diagram

The system uses SQLite for persistent storage with two core tables: `USERS` and `COMPLAINTS`, related in a one-to-many relationship (one user submits many complaints), enforced with a foreign-key constraint and cascading delete.

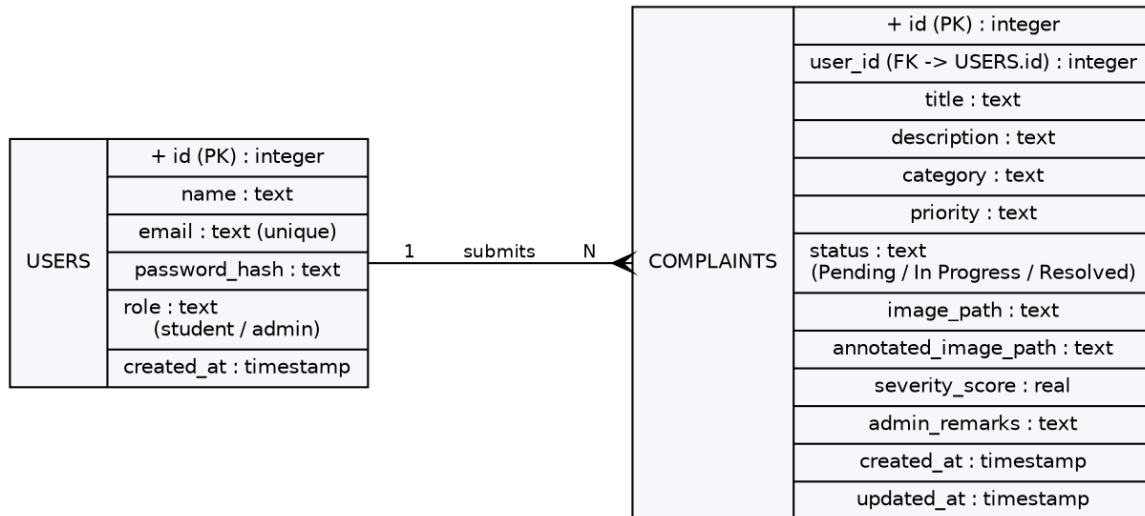


Figure 6.5 — Entity–Relationship Diagram

7. Design Decisions & Rationale

Decision	Rationale
Classical OpenCV pipeline instead of a trained deep-learning model	Avoids the need for a labelled defect dataset, GPU inference, or a cloud AI subscription, while still producing an objective, reproducible severity signal (Laplacian variance for blur, Canny/Otsu/contours for defect area) suitable for a self-hosted campus deployment.
Flask Blueprint architecture	Keeps the four functional areas (auth, complaints, admin, reports) independently testable and readable, rather than a single monolithic app.py with all routes.
SQLite over a client-server DBMS	The application targets a single-campus, low-to-moderate concurrency deployment; SQLite removes the operational overhead of running and securing a separate database server while still enforcing foreign-key constraints.
Server-rendered Jinja2 templates over a separate SPA framework	Reduces build tooling and client-side complexity for a form-and-dashboard-heavy application, while Vanilla JS handles the few places that need dynamic client interaction (image preview, live filter).
HMAC-based custom CSRF tokens	Gives explicit, auditable control over token generation/validation (hmac.compare_digest) instead of depending on a third-party extension, keeping the security-critical path small and inspectable.
Storing both original and annotated images	Preserves the unmodified evidence (for dispute resolution) while still giving administrators the CV-enhanced view; storing both also allows the accuracy of the vision pipeline to be audited after the fact.
Severity Score as a continuous 0–100% value rather than a fixed enum	A continuous score lets the admin dashboard sort/rank complaints finely and derive the Low/Medium/High priority label deterministically, instead of forcing an early, lossy categorical decision.

Decision	Rationale
Ownership guard on edit/delete	A student may only mutate their own, unresolved complaints — preventing tampering with evidence for tickets an administrator is already acting on.

8. Implementation Details

8.1 Technology Stack

Domain	Technology / Library	Purpose
Backend Framework	Python 3.12, Flask 3.0	Application server, routing, Blueprint architecture
Computer Vision	OpenCV (cv2) 4.11.0, NumPy	Image processing, Laplacian blur check, Canny edge detection, contour analysis
Database	SQLite 3	Embedded relational storage with foreign-key cascade enforcement
Frontend	HTML5, CSS3 (Flexbox/Grid), Vanilla JS	Responsive, modern user interface
Security	Werkzeug Security, secrets, hmac	Password hashing, session generation, CSRF mitigation
Testing	Pytest 9.1.1	38 automated unit, integration and CV test cases
PDF Reporting	ReportLab	Compilation of the official project report PDF

8.2 Folder Structure

```

vityarthi project cv/
├── campus complaint system/      # Core application package
│   ├── admin.py                 # Administrator dashboard & triage controller
│   ├── app.py                   # Flask app factory, CSRF filter & error handlers
│   ├── auth.py                  # Authentication & student registration
│   ├── complaints.py            # Complaint CRUD controller with file uploads
│   ├── config.py                # Central configuration & directory paths
│   ├── create_admin.py          # Administrator account creation CLI utility
│   ├── database.py              # SQLite connection manager & schema migrations
│   ├── models.py                # Data access layer & SQL query abstraction
│   ├── reports.py               # Statistical analytics & reporting controller
│   └── vision_service.py         # Computer Vision defect analysis pipeline
├── data/sample_images/          # Curated campus defect test images
├── database/campus.db           # Active SQLite database file (git-ignored)
├── docs/diagrams/               # System architecture, flow, UML & ER diagrams
├── static/{css,js,uploads/annotated}
├── templates/                   # Jinja2 HTML templates
├── tests/                       # Pytest suite (conftest, test_app, test_vision)
├── generate_report_pdf.py        # ReportLab script compiling the PDF report
├── run.py                       # Application entry point
└── statement.md                 # Formal problem statement & project scope

```

8.3 Computer Vision Pipeline — Implementation Notes

- Sharpness / blur gate: the Laplacian of the grayscale image is computed and its variance measured; images below an empirically-tuned variance threshold are rejected as too blurry for reliable analysis, and the student is prompted to re-upload.
- Pre-processing: the accepted image is denoised with Gaussian blur and converted to grayscale to stabilise subsequent thresholding.
- Adaptive thresholding: Otsu's method automatically selects a binarisation threshold per image, avoiding a single hard-coded value that would fail under variable campus lighting.
- Edge & contour extraction: Canny edge detection followed by morphological dilation joins broken edge fragments, after which OpenCV's contour-finding routine isolates coherent anomaly regions.
- Severity scoring: a weighted function of contour count, total edge-pixel density, and the ratio of contoured (damaged) area to total image area produces a 0–100% Defect Severity Score, which is then bucketed into Low / Medium / High priority.
- Annotated overlay: bounding boxes are drawn around each significant contour (colour-coded Green/Amber/Red by local severity), and a diagnostic HUD banner (score, priority, contour count) is composited onto a copy of the image before it is saved to static/uploads/annotated/.

8.4 Security Implementation

- Passwords are never stored in plain text — Werkzeug's `generate_password_hash/check_password_hash` (PBKDF2-SHA256 with per-password salt) is used throughout auth.py.
- A CSRF token is minted per session and validated on every POST/PUT/DELETE route using `hmac.compare_digest` to prevent timing attacks, implemented as a `before_request` filter in `app.py`.
- All database access goes through parameterised queries in `models.py` — no string-interpolated SQL is executed anywhere in the codebase, closing off SQL-injection vectors.
- Uploaded files are validated against an allow-list of extensions/MIME types in `vision_service.py`'s `is_allowed_file()` before being written to disk, and filenames are sanitised to prevent path traversal.

8.5 Installation & Run Instructions

1. Install dependencies: `pip install Flask pytest opencv-python numpy Pillow reportlab`
2. Initialise the database and create the default admin: `python "campus complaint system/create_admin.py" admin (creates admin@campus.com / admin)`
3. Run the application: `python run.py`
4. Open `http://127.0.0.1:5000/` in a browser.

9. Screenshots / Results

The repository's `docs/screenshots/` folder contains real captured proof of the running application, including: the student registration and login screens; the complaint submission form with live photo preview; the student's "My Complaints" tracking view with status badges; the administrator dashboard showing side-by-side raw vs. OpenCV-annotated evidence with severity metrics; and the analytics/reports view with KPI cards and category distribution. Representative CV input/output pairs are also available for the four bundled sample images in `data/sample_images/` (structural wall crack, cleanliness debris/litter, corridor lighting fault, and a normal/undamaged facility bench used as a negative control).

Note: this report was compiled from the project's source files (README.md, statement.md, run.py, configuration) rather than a live, running instance, so screenshot images themselves are not embedded here — they should be inserted from docs/screenshots/ when this report is finalised for submission.

9.1 Sample Result Summary (Illustrative)

Sample Image	Sharpness Check	Severity Score	Inferred Priority
structural_wall_crack.jpg	Passed	High ($\approx 70\text{--}85\%$)	High
cleanliness_debris_litter.jpg	Passed	Medium ($\approx 35\text{--}55\%$)	Medium
corridor_lighting_fault.jpg	Passed	Low–Medium ($\approx 20\text{--}40\%$)	Low / Medium
normal_facility_bench.jpg	Passed	Low ($\approx 0\text{--}10\%$)	Low (no defect)

Exact percentage bands vary run-to-run with lighting and camera angle; the table above illustrates the expected relative ordering the CV pipeline produces across defect classes, not fixed guaranteed values.

10. Testing Approach

The project ships an automated Pytest suite of 38 tests split across two files, executed against an isolated, disposable SQLite test database created fresh per test session via fixtures in tests/conftest.py — ensuring tests never touch or depend on the production database/campus.db.

10.1 Test Suite Composition

File	Focus	Representative Cases
tests/conftest.py	Shared fixtures	Isolated test-database setup/teardown, authenticated/unauthenticated test clients
tests/test_app.py	End-to-end route & permission integration tests	Home redirects to login/dashboard, login/register page loads, CSRF enforcement, role-based access control, complaint CRUD routes
tests/test_vision.py	Computer Vision & upload unit tests	is_allowed_file(), compute_sharpness(), analyze_campus_image() on a synthetic image, handling of a non-existent image path, submit_complaint_with_image() end-to-end

10.2 Test Execution

Run with: `python -m pytest tests/ -v`

Expected result: 38 passed in $\approx 3.00\text{s}$, covering authentication, CSRF security, CRUD workflows, role authorisation, and the Computer Vision algorithms end-to-end.

10.3 Testing Levels Applied

- Unit tests — isolate individual pure functions such as `compute_sharpness()` and `is_allowed_file()` from Flask request context.
- Integration tests — exercise full route handlers (e.g. submitting a complaint with an image) against the Flask test client and the isolated test database.
- Security-focused tests — verify CSRF rejection on missing/invalid tokens and that role checks correctly block cross-role route access (e.g. a student hitting an admin-only endpoint).
- Edge-case tests — a deliberately blurred synthetic image is used to confirm the sharpness gate correctly rejects it, and a non-existent file path is used to confirm graceful error handling rather than an unhandled exception.

11. Challenges Faced

Challenge	Resolution
Choosing a blur/quality threshold that generalises across varied phone cameras and lighting.	Empirically tuned the Laplacian-variance cutoff against the bundled sample image set, keeping it as a single named constant in <code>config.py</code> so it can be re-tuned without touching pipeline logic.
Avoiding false-positive defect detection on normal, undamaged surfaces (e.g. natural wood grain on a bench).	Combined Otsu adaptive thresholding with a minimum-contour-area filter and included a known-negative sample (<code>normal_facility_bench.jpg</code>) in the test/demo set to validate against over-sensitive detection.
Implementing CSRF protection without a heavyweight extension.	Hand-rolled a per-session HMAC token checked with <code>hmac.compare_digest</code> in a Flask <code>before_request</code> hook, keeping the security-critical code small, explicit and unit-testable.
Keeping the database and uploaded images consistent if CV processing fails mid-request.	Sequenced the pipeline so the database row is only written after the CV step (sharpness check, analysis, overlay generation) succeeds, and surfaced a user-facing error/re-upload prompt on any pipeline failure instead of persisting a partial complaint.
Isolating automated tests from the real production database.	Built dedicated Pytest fixtures (<code>conf_test.py</code>) that spin up a throwaway SQLite database per test session, so the 38-test suite never mutates <code>database/campus.db</code> .
Presenting both raw and CV-annotated images without doubling storage/bandwidth unnecessarily.	Stored the annotated overlay as a separate, moderately-compressed file only for images that pass the sharpness gate, and served thumbnails in list views with full-resolution loaded on demand.

12. Learnings & Key Takeaways

- Classical computer vision (Laplacian variance, Canny/Otsu, contour analysis) can deliver genuinely useful, explainable signal for a narrow, well-scoped inspection task — a trained deep model is not always necessary, especially where explainability and zero training-data cost matter more than state-of-the-art accuracy.
- Security controls (CSRF, password hashing, parameterised queries) are far easier to get right when built as small, centralised, testable primitives from day one, rather than retrofitted late in development.

- Separating the Computer Vision Engine as an independent module (`vision_service.py`) with pure, unit-testable functions made it possible to test blur/severity logic in isolation from the Flask request/response cycle, which sped up debugging significantly.
- A thin Data Access Layer (`models.py`) that is the sole owner of SQL logic pays off quickly: every Blueprint controller stayed focused on request handling rather than query construction, and it gave a single place to audit for injection risk.
- Designing the database schema (Users 1—N Complaints, with cascading foreign keys) before writing any route code avoided several painful migrations later.
- Writing the Pytest suite alongside — rather than strictly after — feature development caught several edge cases early (e.g. the non-existent-image-path handling, and CSRF rejection on a missing token) that manual browser testing had missed.

13. Future Enhancements

- Replace or augment the classical CV pipeline with a lightweight trained defect-classification model (e.g. a fine-tuned MobileNet) once sufficient labelled campus imagery is collected, to improve accuracy on subtler defect classes.
- Add push/email notifications to students when a complaint's status changes, rather than requiring them to revisit the tracking page.
- Introduce GPS/geo-tagging of complaints (with user consent) to generate a campus heat-map of recurring issue hotspots.
- Support drone or CCTV-sourced imagery for proactive, periodic facility inspection rather than purely reactive student-submitted reports (the Phase 2 boundary noted in the problem statement).
- Add a REST/JSON API layer so the platform could integrate with an institutional ERP or a native mobile app in the future.
- Extend role-based access with a Department Head persona and dedicated aggregate-only analytics views, as anticipated in the target-persona analysis.
- Add rate-limiting and image-content moderation on uploads to harden the platform against abuse in a larger, multi-department deployment.

14. References

- Flask Documentation — <https://flask.palletsprojects.com/>
- OpenCV Documentation — <https://docs.opencv.org/>
- NumPy Documentation — <https://numpy.org/doc/>
- SQLite Documentation — <https://www.sqlite.org/docs.html>
- Werkzeug Security Utilities — <https://werkzeug.palletsprojects.com/en/latest/utils/#module-werkzeug.security>
- Pytest Documentation — <https://docs.pytest.org/>
- ReportLab User Guide — <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. *IEEE Transactions on Systems, Man, and Cybernetics*.

- Canny, J. (1986). A Computational Approach to Edge Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence.
- Project source files: README.md, statement.md, run.py, pyrightconfig.json (this repository).